

NAME: Santillan, Ralph Lawrence S.

CSIT340-G3

Questions

1. Why is the request list stored in state rather than a ref?

- The request list affects what appears on screen: queue cards, counts, filters, and empty states. State updates trigger a re-render, so the UI stays synchronized. A ref can store changing data without re-rendering, so it would not be appropriate for the main request list.

2. Why must existing state arrays and objects remain unmodified?

- React relies on changed references to detect updates efficiently. If an existing array or object is changed directly, React may not recognize that anything changed, which can cause stale or inconsistent UI. Immutable updates also make the code easier to reason about and help prevent accidental side effects. For example:

```
setRequests(previous => [...previous, newRequest]);
```

creates a new array instead of modifying

3. When should you use a functional state update?

- Use a functional update when the new state depends on the previous state. This application uses it when adding, resolving, or deleting requests:

```
setRequests(previous =>
  previous.map(request =>
    request.id === requestId
      ? { ...request, status: "Resolved" }
      : request
  )
);
```

This ensures React uses the most recent state, even when multiple updates happen close together.

4. What does each effect dependency array control in this application?

- `useEffect(..., [requests])`

Saves the request list to `localStorage` whenever the requests change.

- `useEffect(..., [waitingCount])`

Updates the browser title whenever the number of waiting requests changes.

- `useEffect(..., [theme])` in `ThemeContext`

Saves the selected theme and updates the document's `data-theme` attribute whenever the theme changes.

- `useEffect(..., [])` in `RequestForm`

Focuses the student name input once when the form first mounts.

5. Why is an input ref appropriate for focus management?

- Focus is a direct browser interaction, not display data. A ref provides access to the actual input DOM element without causing a re-render. The form uses it to focus the name field when the component loads, after a successful submission, or when validation fails.

6. How does context reduce passing props through intermediate components?

- The `ThemeContext` makes the current theme and `toggleTheme` function available directly to components such as `Header`, `RequestForm`, and `RequestCard`. Those values do not need to be passed through `App` and other intermediate components as props. This avoids unnecessary prop drilling.

7. Why are filtered requests and queue counts calculated during rendering?
- They are derived directly from the current requests state and selected filter.
- Calculating them during rendering keeps one source of truth and prevents duplicated state from becoming out of sync.

For example:

- o Waiting count comes from requests whose status is "Waiting".
- o Resolved count comes from requests whose status is "Resolved".
- o Filtered requests come from the selected filter.
- o Sorted requests are derived from the filtered list.